1 - Computing Infrastructures Fundamentals

Computing Infrastructures

Unat Teksen

April 3, 2026

1 What is a Computing Infrastructure?

Definition

A **computing infrastructure** is a technological infrastructure that provides **hardware** and **software** for computation to **other systems and services**.

- **Hardware** — physical machines, processors, storage, network cards
- **Software** — operating systems, middleware, virtualization layers
- **Serves others** — exists to support applications/services running on top of it; it is a *platform*

2 The Spectrum of Computing Infrastructures

All types are compared relative to a **PC = 1x** (reference point) on two axes: **RAM** and **Computation Speedup**.

maingreen!20 **Type**	**Computation**	**RAM**	**Primary Use**
Internet of Things	0.0001x–0.0005x	0.01x–0.05x	Sensing, actuating
Embedded Devices/PCs	0.05x–0.1x	0.1x–0.5x	Local intelligence
PC (reference)	**1x**	**1x**	General purpose
Edge Computing Systems	20x–50x	50x–100x	Local heavy compute, AI
Data Center	100x–1000x	Very high	Training, large-scale

Key Takeaway

These types are **not competing** — they form a **hierarchy**: IoT collects → Edge processes locally → Cloud stores and learns → Models deployed back to Edge.

3 Data Centers

3.1 Physical Structure

A data center houses servers organized for three functions:

- **Server for Processing** — CPUs, GPUs doing computation
- **Server for Storage** — databases, file systems

- **Server for Communication** — networking equipment, data flow

3.2 Advantages and Disadvantages

Advantages:

- Lower IT costs
- High performance
- Instant software updates
- "Unlimited" storage capacity
- Increased data reliability
- Universal document access
- Device independence

Disadvantages:

- Requires constant internet connection
- Poor with low-speed connections
- Limited hardware features
- Privacy and security issues
- High power consumption
- **Latency in making decisions** ← key

3.3 Costs and Environmental Impact

Amortized Cost Breakdown:

maingreen!20 **Component**	**% Cost**	**Sub-components**
Servers	∼45%	CPU, memory, disk
Infrastructure	∼25%	UPS, cooling, power distribution
Power draw	∼15%	Electrical utility costs
Network	∼15%	Switches, links, transit

- **Installed power:** ∼30 GW (2010) → ∼60 GW (2020), growing rapidly due to AI
- **Water:** a 15 MW data center uses ≈130 million gallons/year (= 3 hospitals)
- **Electricity (US):** 1.9% of total in 2014 → projected up to 12% in future

Definition

UPS (Uninterruptible Power Supply): Battery systems that keep servers running during a power grid failure until generators start.

4 The Data Center Dilemma: Build, Rent, or Cloud?

Definition

Data centers are always part of a business. Not all money should be spent on the datacenter — building one is not the only option.

4.1 Option 1: Enterprise Data Center (Build Your Own)

- Facility wholly built, maintained, operated, and managed by the company
- **Pros:** Company asset, complete control of operational activities
- **Cons:** Very high capital investment, requires specialized expertise to design and run

Typical requirements: secure site, redundant power/cooling, raised-floor, fire suppression, complex networking, centralized monitoring.

4.2 Option 2: Colocation Data Center

- Large facility built to **accommodate multiple businesses**; company rents space (cabinet/cage/room)

- Can use own IT equipment or rent from the co-locator
- **Pros:** Still high control on IT equipment, no DC design skills needed, lower capital investment, shared costs for high-quality environment
- **Cons:** DC is no longer a company asset, restricted maintenance access, IT equipment purchase still your responsibility

Definition

99.995% uptime (Five Nines): ≈26 minutes of downtime per year — a common colocation provider guarantee.

4.3 Option 3: Managed Hosting / Third Party DC

- DC managed by a third party; company rents IT equipment and infrastructure
- Resources are **not shared** — still a **single-tenant solution**
- **Pros:** Low capital investment, cost savings for small businesses, no DC expertise needed
- **Cons:** Higher operational costs, lower control, **customer lock-in problem**

4.4 Option 4: Cloud Computing

Cloud solutions are **additional options** on the table beyond physical infrastructure.

maingreen!20 **Model**	**What YOU manage**
On-Premises IT	Everything: Data, Apps, DBs, OS, Virtualization, Physical Servers, Storage, Networks, DC
Colocation	Data, Apps, DBs, OS, Virtualization, Physical Servers
Hosting	Data, Apps, DBs, OS
IaaS	Data, Apps, DBs, OS
PaaS	Data, Apps
SaaS	Data only

- **IaaS** (e.g., AWS EC2): you get VMs, install your own OS and apps
- **PaaS** (e.g., Google App Engine): you deploy code, platform manages OS/runtime
- **SaaS** (e.g., Gmail): you just use the application

Exam Focus

- Know **all four deployment options** and pros/cons of each
- **Cloud spectrum**: On-premises → Colocation → Hosting → IaaS → PaaS → SaaS
- **Five Nines** = 99.995% uptime ≈ 26 min downtime/year
- **Cost split**: 45% servers, 25% infrastructure, 15% power, 15% network
- **Latency** is the key disadvantage driving Edge Computing

5 Virtual Machines and Containers

5.1 Virtual Machine Monitors (VMMs / Hypervisors)

Definition

VMM/Hypervisor: Software layer between physical hardware and Guest OSes. Provides the full stack (OS, LIB, APP). Applications depend on Guest OS.

Stack: `Infrastructure` → `Host OS` → `Hypervisor` → `[Guest OS | Bins/Libs | App]` ×`N`

- Each VM gets its own complete OS — fully isolated
- **Pros:** Strong isolation, full OS flexibility
- **Cons:** Heavy overhead, slow startup (minutes), high RAM usage per VM

5.2 Containers (e.g., Docker)

Definition

Container: Applications packaged with all their dependencies into a standardized unit. **Share the host OS kernel** — no Guest OS per application.

Stack: `Infrastructure` → `Host OS` → `Docker Engine` → `[Bins/Libs | App]` ×`N`

- **Pros:** Lightweight, fast startup (seconds), low overhead
- **Cons:** Less isolation — kernel vulnerability affects all containers

5.3 VM vs. Container Comparison

maingreen!20 **Feature**	**VM**	**Container**
OS per app	Yes (Guest OS)	No (shared host OS)
Isolation	Strong	Moderate
Startup time	Minutes	Seconds
Overhead	High	Low
Use case	Full OS flexibility	Microservices, deployment

Exam Focus

- Know **both stack diagrams** and be able to explain each layer
- **Key difference:** VMs have Guest OS; containers share host OS kernel
- Know trade-offs: isolation vs. overhead, startup time

6 Edge / Fog Computing Systems

Definition

Edge Computing: Bringing computation **physically close to where data is generated** to minimize latency. Motivated by the latency disadvantage of centralized data centers.

6.1 Three-Layer Architecture (Pyramid Model)

maingreen!20 **Layer**	**Role**	**Timescale**
Cloud (top)	Virtualized compute/storage, model training, analytics	Minutes–hours
Edge/Fog Servers (mid)	On-premises heavy compute, AI inference, local processing	100ms–minutes
IoT & Edge Sensors (bottom)	Data acquisition, partial processing, sensing/actuating	μsecs–msecs

Key design pattern: *Learning in the Cloud, Deploying in the Fog.*

Real Example — Wind Turbine Inspection:

- **Cloud solution:** Manual upload → cloud processes → results later. No real-time feedback to UAV operator.
- **Edge solution:** On-site device immediately assesses image quality and detects blade damage, giving real-time operator feedback. Cloud only handles final classification and reporting.

6.2 Edge/Fog Hardware (Dell PowerEdge Examples)

- Server-class machines with Intel Xeon processors, up to 8 TB RAM, NVMe storage
- Heavy GPU presence (NVIDIA H100, A100) — edge increasingly means **running deep learning on-site**

Definition

Rack Unit (U): Standard height unit for rack-mounted equipment. 1U = 44.45 mm. A standard rack = 42U. Determines deployment density.

6.3 Advantages and Disadvantages

Advantages:

- High computational capacity
- Distributed computing
- Privacy and security
- Reduced latency

Disadvantages:

- Requires power connection
- Requires cloud connectivity

Exam Focus

- **Why Edge Computing exists:** latency — centralized DC cannot respond fast enough
- Know the **three-layer pyramid** and timescales at each level
- **Design pattern:** Learning in Cloud, Deploying in Fog

7 Embedded PCs

Definition

Embedded PCs: Full computers on a single board, designed for pervasive embedded deployment (machines, robots, vehicles). Run full OS (typically Linux). Bridge between IoT and edge servers.

Examples from slides:

- **Raspberry Pi 5** — 8 GB RAM, Quad-Core ARM A76, 2.4 GHz. Full Linux, general-purpose embedded computing.
- **Nvidia Jetson Orin Nano** — 8 GB, 6×1.5 GHz. Dedicated neural network acceleration for AI at the edge.
- **BeagleBone Black** — ARM Cortex A8, 512 MB RAM. Hardware-oriented with many GPIO pins for industrial control.
- **Google Coral Dev Board** — 1 GB RAM + **TPU** (Tensor Processing Unit) for low-power TensorFlow Lite inference.

Advantages:

- Pervasive computing
- High performance for size
- Development boards available
- Programmed like PCs
- Large community

Disadvantages:

- High power consumption for battery use (RPi: 5–8W)
- Some HW design required

8 Internet of Things (IoT)

Definition

An **IoT device** is any everyday object embedded with **sensors, software, and internet connectivity** to collect and exchange data with other devices, with **limited need to process and store data**.

Examples from slides: Arduino (ATmega328), STM32 (ARM Cortex-M), ESP32 (WiFi/BT built-in), Particle Argon.

8.1 Arduino Nano Family Specs

maingreen!20 **Device**	**MCU**	**Clock**	**SRAM**	**Flash**	
Arduino Nano	ATmega328	16 MHz	2 kB	32 kB	
Nano ESP32-S3	ESP32-S3	up to 240 MHz	512 kB	16 MB	
Nano 33 BLE	nRF52840	64 MHz	256 kB	1 MB	
Nano RP2040	RP2040	133 MHz	264 kB	16 MB	
Nano Every	ATMega4809	16 MHz	6 kB	48 kB	

Definition

- **SRAM:** Volatile working memory (lost on power off) — like RAM in a PC. Very scarce in IoT.
- **Flash:** Non-volatile program storage (survives power off) — where code lives.

Advantages:

- Highly pervasive
- Wireless connection
- Battery powered
- Low cost
- Sensing and actuating

Disadvantages:

- Low computing ability
- Energy constraints
- Memory constraints (kB scale)
- Difficult to program (no OS, no memory protection)

Exam Focus

- **Arduino RAM: 2 kB; Raspberry Pi: 8 GB = 16,000× more** — know this scale difference
- **SRAM** = volatile working memory; **Flash** = non-volatile program storage
- IoT devices are **collectors, not processors** — limited need to process/store data

9 The IT Perspective for Computing Infrastructures

9.1 Three-Layer IT Model

Every computing infrastructure can be described with three layers:

maingreen!20 **Layer**	**Description**	**In Cloud Context**
Software (Application)	The app — the reason the system exists	Cloud-Based Apps, X-Learning-as-a-Service
Software (Environment)	Programs/libraries to control HW and build apps (OS, hypervisor, middleware)	Cloud (Virtualization, IaaS/PaaS/SaaS)
Hardware	Physical resources: compute, storage, I/O, networking	Datacenter (servers, storage, network)

Definition

X-Learning-as-a-Service: Accessing machine learning capabilities via cloud APIs without managing any infrastructure (e.g., OpenAI API, Google Vision API, AWS Rekognition).

Key Takeaway

This three-layer model applies at every scale: Arduino (MCU / bare-metal runtime / sensor loop), PC (Intel CPU / Linux / browser), Data Center (server rack / hypervisor+VMs / cloud service). It is the mental model for the **entire course**.

Master Exam Summary

Exam Focus

Key Numbers:

- Cost split: **45% servers, 25% infrastructure, 15% power, 15% network**
- Five Nines: **99.995% uptime ≈ 26 min downtime/year**
- 15 MW DC water use: **130M gallons/year**
- Global power: **30 GW (2010) → 60 GW (2020)**
- Arduino SRAM: **2 kB**; RPi RAM: **8 GB = 16,000× difference**

Must-Know Comparisons:

- **VM vs. Container** — Guest OS vs. shared host OS; isolation vs. overhead
- **Enterprise vs. Colocation vs. Managed Hosting vs. Cloud** — pros/cons of each
- **IaaS vs. PaaS vs. SaaS** — what you manage vs. what provider manages
- **IoT vs. Embedded PC vs. Edge Server** — compute, RAM, use cases

Must-Know Concepts:

- Why Edge Computing exists: **latency** of centralized DCs
- Three-layer pyramid: **IoT** → **Fog/Edge** → **Cloud** with timescales
- Three-layer IT model: **HW** → **SW Environment** → **SW Application**
- Design pattern: **Learning in Cloud, Deploying in Fog**
- **Lock-in problem** in managed hosting
- **SRAM** (volatile) vs. **Flash** (non-volatile) in IoT